

One Namespace, or One **Vote** Each

Thirteen cards from nine runnable topologies. Cards 01 to 06 build the words, simple first: the map, Raft, placement, accounts and domains. Cards 07 to 12 use those words on each shape. Card 13 is what this lab got wrong and corrected.

THE QUESTION EVERY CARD SERVES

Two regions, one platform. **Do you want one global stream name, or do you want each region to keep working when the other goes dark?** With two sites, no shape in this lab gives both. A third site that votes keeps one meta group alive when either region is dark (card 08). A stream stored in the dark region is still dark with it.

FOUR SENTENCES THE DECK DEPENDS ON

A **gateway** joins clusters into one supercluster with **one meta group**. A **leaf link**, as this lab builds it — **\$SYS** not carried, a domain per side — leaves a meta group on each side. An **account** is the wall for data. A **domain** names a JetStream; it is not a wall.

CARD FAMILIES

- Map — the topologies and the words
- Quorum — Raft, three groups, arbiters
- Placement — where a stream lands
- Boundaries — accounts, domains
- Links — gateway, leaf node
- Copying — mirrors, export / import
- Traps — both links at once
- Retractions — what we got wrong



KEY

Every card is drawn to the report's rules

The figures on these cards use the **same drawing grammar as `REPORT.html`** . A boundary means the same thing in both. If a card and the report disagree, the report is right — it is generated from the run.

BOUNDARIES AND LINKS

JETSTREAM DOMAIN — A NAME

ACCOUNT — THE WALL FOR DATA

orange dashed — jetstream.domain. Names one JetStream and sets its API prefix `$JS.<domain>.API`. Not a wall. It does not create a meta group by itself (card 06).

ACCOUNT LB_ZA

CLUSTER za — REGION za

yellow dashed — an account. Two accounts cannot see each other's subjects, streams or consumers without an export and an import.

gateway — one supercluster, one meta group

leaf — in this lab, a meta group each side

mirror — a copy somebody wrote down

kill -STOP — how every outage here is made

Each figure ends with the check IDs it draws. A claim with no ID carries a tag: (documented), (inferred) or (proposed).

FOUR KINDS OF STATEMENT — EACH ONE IS TAGGED	
TAG	WHAT IT MEANS
A11 MEASURED	This rig ran it and the answer matched. The check ID points at one row of <code>REPORT.md</code> . An ID with a small letter, such as <code>A11a</code> , is a recorded observation, not a pass or fail.
DOCUMENTED	The NATS documentation says it. This lab did not test it. The source is on the provenance page.
INFERRED	It follows from measured checks and the documented rules, but no check asks it directly.
PROPOSED	A plan, such as a migration step. Nobody has run it.

TWO THINGS THE DRAWINGS WILL NOT LET YOU ASSUME

A domain box is not an account box.

A frozen region is `kill -STOP` , not a severed link.

They are different colours because they are different kinds of boundary. A domain separates *names*. An account separates *data*, and nothing crosses it without an export.

In a real WAN partition both sides stay alive and cannot reach each other. The lab did not build that rig. Where a card draws a live cut, the cut is tagged INFERRED.

01

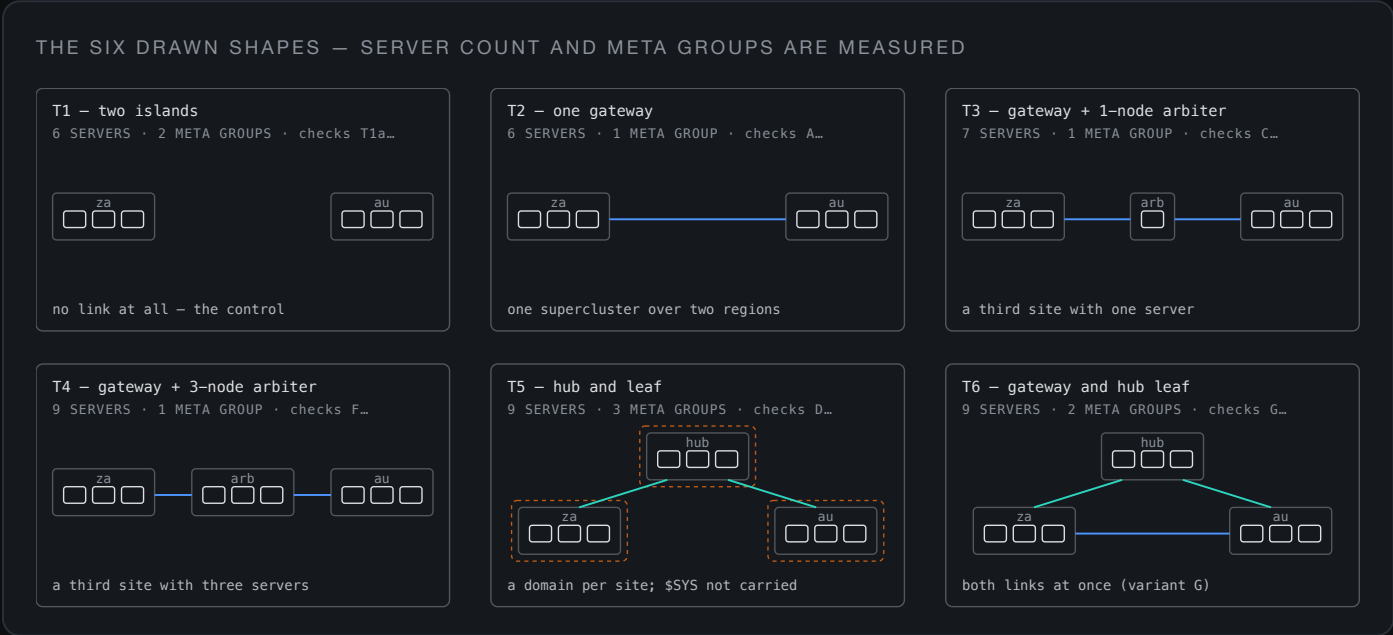
MAP — THE NINE TOPOLOGIES

Five shapes, four variants, one letter each

Demo 03 built **nine runnable topologies** from nothing and measured each one: five shapes, **T1 to T5**, and four variants, **B, E, G and H**. G is also called **T6**. Every check ID starts with the letter of the rig that ran it.

DECISION

Read a check ID as a place: **A11** is check 11 on rig A, which is T2. Before you trust a claim, find its letter here and look at the shape it was measured on.



THE FOUR VARIANTS — EACH CHANGES ONE THING AND ASKS ONE QUESTION

VARIANT	WHAT IT CHANGES	CARD
B · on T2	A different <code>jetstream.domain</code> on each gateway-joined cluster. Does a domain split the meta group?	06
E · on T2	Two accounts, and an export / import between them. Can a region share data on purpose, across the account wall?	11
G · T6	A gateway and a hub leaf link. Runs twice: phase 1 with no domain, phase 2 with a domain per site.	12
H · on T5	An account per region carried over the leaf link, beside the shared account. Does that stop double capture?	09

MECHANISM — THE WORDS, SIMPLE FIRST		
WORD	WHAT IT MEANS	IN THIS LAB
Server	One <code>nats-server</code> process.	6, 7 or 9 per rig, all on one laptop.
Cluster	Servers joined by routes. They act as one.	One per region: <code>za</code> , <code>au</code> . Plus <code>arb</code> or <code>hub</code> .
Gateway	Joins clusters into one supercluster .	T2, T3, T4, T6.
Leaf link	A server or cluster connects to another as a leaf. Each remote binds one account.	T5, H, T6. Binds <code>LB</code> (H adds <code>LB_ZA</code> / <code>LB_AU</code>). Never <code>\$SYS</code> .
Account	Its own subjects, streams and consumers. The wall for data.	<code>LB</code> shared; <code>LB_ZA</code> , <code>LB_AU</code> per region; <code>\$SYS</code> the system account.
JetStream	The storage layer: streams, consumers, KV. One JetStream is the set of servers that share one meta group.	Enabled on every server.
Meta group	The Raft group of every server in one JetStream. It holds the catalogue — which streams and consumers exist, and where. Its leader handles create, update and delete.	1 on T2–T4, 2 on T1 and T6, 3 on T5 and H.
Domain	<code>jetstream.domain</code> : a name for one JetStream, and its API prefix <code>\$JS.<domain>.API</code> .	None on T1–T4 and G phase 1. <code>hub</code> / <code>za</code> / <code>au</code> on T5, H, G phase 2. B: one per cluster.
JetStream namespace	Where a stream name must be unique: one meta group × one account .	Card 05.

HELD STILL ON EVERY RIG

› `nats-server` 2.14.6 and the `nats` CLI 0.4.0, on one host.

› JetStream on every server. Accounts are user / password pairs in hand-written `.conf` files. No operator trust chain.

› `$SYS` is never carried over a leaf link. This one fact shapes cards 06, 09 and 12.

› Every outage is `kill -STOP` on whole servers, proved dark before it is measured.

NOT TESTED ON ANY RIG

› A real WAN: latency, packet loss, a live two-sided partition.

› A leaf link that **carries** `$SYS` . The docs say it changes the result `DOCUMENTED` . Card 06.

› A real operator (`nsc`) trust chain, or JWT accounts.

› Mirror lag, throughput and any production timing.

02

QUORUM — RAFT

A vote needs more than half, and servers vote, not sites

JetStream keeps its state in **Raft groups**. A Raft group is a set of servers that elect one **leader**. The leader can act only while **more than half** of the group can hear it. That number is the **quorum**, or majority.

DECISION

Before you pick a shape, count servers. Ask: **when one site goes dark, is more than half of the group still alive?** If not, that group stops. Sites do not vote; servers do.

MAJORITY, THEN A DARK SITE — GREEN IS LIVE, RED IS DARK, GREY IS THE REST

A MAJORITY IS MORE THAN HALF: $\text{FLOOR}(N / 2) + 1$ (DOCUMENTED)

3 servers



majority 2 — T1b T1c D2

6 servers



majority 4 — A2

7 servers



majority 4 — C1

9 servers



majority 5 — F1

A SITE GOES DARK — COUNT THE SERVERS THAT ARE LEFT

T2 · ZA dark



3 of 6, below 4 — no leader — A10

T4 · AU dark



6 of 9 — a leader, creates — F3 F4

T4 · AU + 1 arb



5 of 9, exactly 5 — still elects — F5 F6

T4 · 4 of 9 left



4 of 9 — no leader, create fails — F7 F8

T5 · AU dark



3 groups of 3 — ZA counts alone, 3 of 3 — D24 D25

a live WAN cut (inferred, not built): a 3 | 3 split leaves no side a majority of 6.

a 6 | 3 split leaves the side of 6 a majority of 9. two sides can never both hold a majority of one group.

MECHANISM

› Raft keeps one ordered log per group. The leader adds an entry; the entry is **committed** when a majority has stored it. The docs give quorum as half the cluster size, plus one **DOCUMENTED**.

› A majority is always **more than half**. So two groups of servers that cannot reach each other can never both hold a majority of the **same** group. One side, at most, keeps a leader.

› An **uneven** split can leave one side a majority: 6 of 9 keeps a leader (**F3**). An **even** split of an even group leaves no side one: 3 of 6 has no leader (**A10**).

› Every dark site in this lab is **kill -STOP** . A live WAN cut, where both sides run and cannot talk, uses the same arithmetic, but it was not built **INFERRED**.

SLACK — WHAT EACH SHAPE CAN STILL LOSE AFTER ONE REGION IS DARK

SHAPE	SERVICES IN THE GROUP	MAJORITY LEFT, ONE REGION DARK	SLACK
T1, T5 each site	3	2 3 — the other site is not in this group T1g D24	1 server INFERRED
T2 gateway	6	4 3 — no leader	none A10
T3 + 1-node arbiter	7	4 4 — exactly the majority	none C12
T4 + 3-node arbiter	9	5 6	1 server F11 F5
T6 ZA + AU group	6	4 3 — no leader	none G10

PRO

› **The arithmetic is exact.** Count the live servers in a group and you know if it can act. Every leader and no-leader result above was measured.

› At most one side keeps a leader. Raft does not let two leaders change the same group.

CON

› **A group that spans two sites stops when either site goes dark**, if the sites are equal.

› The **leader** field in **/jsz?meta=1** stays stale for about a minute after a majority dies. The rig polls for it; a person reading it once will be misled.

03

QUORUM — THREE KINDS OF RAFT GROUP

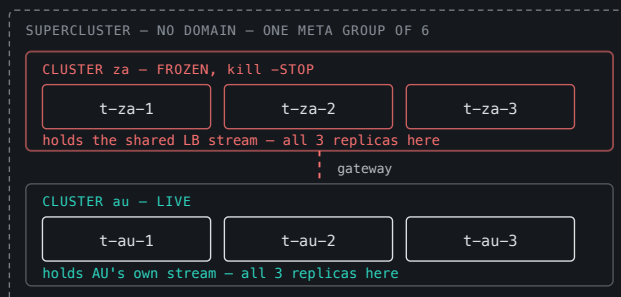
"Can we still write?" is three questions

JetStream runs **three kinds of Raft group**: one meta group, one group per stream, and one group per consumer. Each counts its own majority. So in a gateway shape with a region dark, **one question fails and the others can still work**.

DECISION

Before you accept any topology, say **which of the three you need during an outage**. Losing the meta group is a **change** freeze, not a **write** freeze. They are not the same outage.

ONE RIG, THREE ANSWERS — T2 GATEWAY, ZA FROZEN, AU LIVE



1 — CREATE A STREAM · the META group needs a majority of all 6 servers; 3 of 6 is below 4 fails, 10008 — A11 A11a

2 — PUBLISH TO AU'S STREAM · the STREAM group needs only that stream's own 3 replicas, all in au 1 message stored — A12

3 — READ · the stream, through a CONSUMER

AU's own stream: ok — A20

the shared stream in za: fails, 10008 — A21 A21a

a consumer lives with its stream — A18 A19

a plain nats pub is a CORE publish. it exits 0 whether the message was stored or not.

the only proof is counting the stream afterwards — A12 F21 D23 G12 H16

MECHANISM — THREE KINDS OF GROUP, THREE MAJORITIES

GROUP	HOLDS	ITS SERVERS
Meta — one	The catalogue: which streams and consumers exist, and where. Create, update and delete go through it.	Every server in one JetStream. 6 on T2.
Stream — one each	The messages of one stream. A publish is stored when a majority of its replicas has it.	Its replicas: 1, 3 or 5 DOCUMENTED , all in one cluster (F17).
Consumer — one each	One consumer's position: what was delivered and acked.	With its stream A18 A19 .

>

The docs name the three kinds and say each stream and each consumer is its own Raft group **DOCUMENTED** .

>

A read of the dark region's stream fails and a read of the live region's stream works, on the same rig, at the same moment (**A20** , **A21**). So the read follows the stream's own replicas, not the meta group **INFERRED** . The error that arrives is still **10008** (**A21a**).

>

This split is why **D03-R1** in the report is three columns, not one.

THE THREE QUESTIONS AGAINST EVERY SHAPE MEASURED

SHAPE	CREATE, WITH A REGION DARK	PUBLISH INTO A LOCAL STREAM	READ, WITH A REGION DARK
T1 no link	yes T1g T1h	not measured	not measured
T2 gateway	no — 10008 A11	yes A12	own: yes A20 dark region's: no A21
T3 + 1-node arbiter	yes C4 C6	not measured	not measured
T4 + 3-node arbiter	yes F4 F23	yes F21 F24	not measured
T5 hub and leaf	yes D25	yes D26	not measured
H T5, account per region	not run	not run	not run
T6 both links at once	no — 10008 G11	yes G12	not measured

PRO — OF ASKING IT THREE WAYS

>

You learn that a gateway outage is a **change freeze**, not a full stop.

>

You can size the risk properly: creation stops, existing traffic in the live region carries on.

>

It tells you exactly which checks to demand of any claim somebody makes.

CON — THE CATCH IN ROW TWO

>

"Publish keeps working" holds on **one condition**: that stream's own replicas must still hold their own majority.

>

Every outage here darkens a whole region, so the surviving streams kept all three replicas at home.

>

Losing a single stream's own replicas is a different question, and is not measured here.

VERDICT

Write the outage sentence down before choosing a shape: "**during a region outage we must still be able to <create / publish / read>.**" A11, A12, A20, A21 · Card 03 · 2 of 2

04

PLACEMENT — WHERE A STREAM LANDS

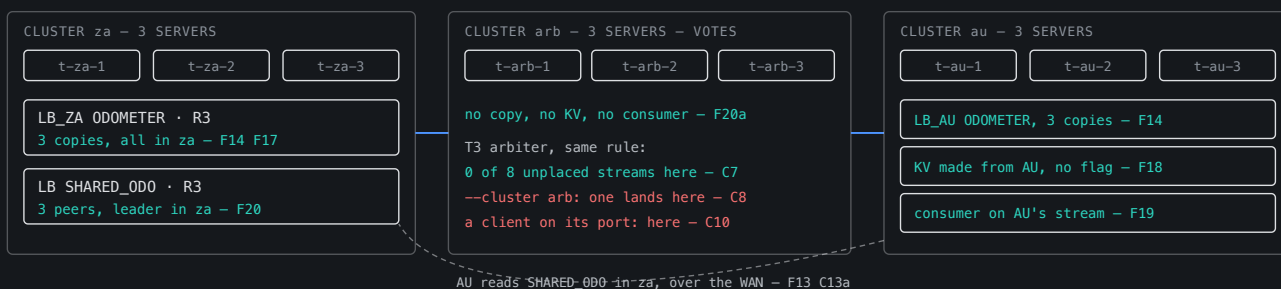
Placement decides where bytes sit, not who survives

Every stream is placed **once, when it is created**, into **one cluster**. Ask for nothing and it lands where the client is. Name a cluster and it lands there. T4 is the reference here: three clusters, one meta group of 9.

DECISION

For every stream, KV bucket and consumer, **say which cluster it belongs in**, and make the create call say so. A stream stored in the dark region is dark with it, whatever the quorum does.

T4 — WHAT LANDED WHERE, WITH NO REGION DARK



PLACEMENT IS PER STREAM, AT CREATE TIME

no flag → the cluster of the server the client is connected to — F18 A9 C18
--cluster X → cluster X, from any client — E13 E14 on T2
every replica of one stream sits in one cluster — F17 C17
that cluster must hold R servers, or the create fails with 10005 — C9 B4

— on T4 (inferred)

MECHANISM

- Placement belongs to a stream, not to an account. Two streams in one account can sit in two clusters: `--cluster za` and `--cluster au` from the same client both work (E13 , E14).
- It is decided **once, at create time**. A stream config can name a cluster or tags `DOCUMENTED`. This lab tested `--cluster` only; tags were not tested.
- With no placement, the stream lands in the client's own cluster. The meta leader records the choice; **where the meta leader sits does not pick the cluster** `INFERRED` — no check moved the leader to test it.
- Placement is not quorum**. T4 survives a dark region because of the meta majority (card 02). A stream placed in that dark region is still dark.
- Control path and data path differ**. A read from AU of a stream in za gives `1 / 1` (F13 , A26): that is **one** stream, read over the WAN (C13a), not two copies. A second copy in au needs a mirror (A15 , card 10).

WHAT YOU ASK, AND WHERE IT LANDS

YOU ASK	LANDS IN	PROOF
no flag, from a ZA client	za — never the arbiter	C7 (T3, 8 streams)
no flag, KV bucket from an AU client	au	F18 A9 C18
<code>--cluster za</code> / <code>au</code> , same client	as named	E13 E14 (T2) · T4 <code>INFERRED</code>
a mirror, placed in the other region	au	A15
<code>--cluster arb</code> , R1	arb — unreplicated	C8 (T3)
any create from a client on the arbiter's port	arb	C10 (T3)
<code>--cluster arb</code> , R3, one-node arbiter	fails, 10005	C9
the blind cluster of variant B	fails, 10005	B4 , card 06

PRO

- Predictable**. Local by default, explicit when named. Every row above landed where the rule says.
- Replicas stay in one cluster (F17), so a dark region never holds half a stream.

CON

- A shared-account projection can split**. Its consumer lands with a ZA stream while its KV, made from AU, lands in au (A19 , F18).
- A consumer on a far stream pays the WAN on every pull (A19).
- The arbiter stores whatever is placed on it (C8 , C10). A 3-node arbiter could even take an R3 stream `INFERRED`.

05

BOUNDARIES — ACCOUNTS AND THE JETSTREAM NAMESPACE

A stream name is unique in one meta group and one account

Two things decide whether two regions can both own a stream called **ODOMETER**: how many meta groups there are, and whether they share an account. One meta group × one account is one JetStream namespace. Only there does a name clash.

DECISION

Give each region **its own account** unless you truly want one name for both regions. Then pick the link. The account choice works the same on every shape; the link choice does not.

TWO CHOICES, FOUR OUTCOMES — ONE PUBLISH IN ZA, THEN COUNT

ONE META GROUP
joined by a gateway
T2, T3, T4, and
T6's two regions

SHARED ACCOUNT LB — BOTH REGIONS

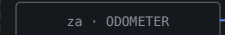
LB — ONE NAMESPACE



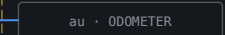
a 2nd ODOMETER is refused, 10058 — A4 B8 G4
1 / 1 = ONE copy, read from both sides — A26 G7a
AU's stream info hands back za's stream — A5

AN ACCOUNT PER REGION — LB_ZA, LB_AU

LB_ZA



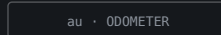
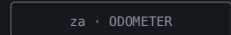
LB_AU



each creates its own ODOMETER — A6 A7 E7 E8
1 / 0 — the other account sees nothing — A8
two namespaces on one meta group

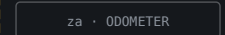
A META GROUP
PER SITE
no link, or this
lab's leaf link
T1, T5, H

LB — A NAMESPACE PER SITE

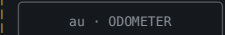


both create ODOMETER, no 10058 — T1d D3
T1, no link: 1 / 0 — T1f
T5, LB over the leaf: 1 / 1 STORED, TWO copies — D5 D5a

LB_ZA



LB_AU



1 / 0 — nothing is shared — D18 H6
a shared hub is not a shared account — H6a

ONE META GROUP × ONE ACCOUNT = ONE JETSTREAM NAMESPACE. A NAME IS UNIQUE ONLY INSIDE ONE.

MECHANISM

>

An **account** owns its subjects, streams and consumers. Nothing crosses to another account without an export and an import (card 11).

>

A **meta group** keeps one catalogue. Inside it, one account can hold one stream per name. So the unit where a name must be unique is **one meta group × one account**: the JetStream namespace.

>

A **domain name does not split it**. Variant B put two domain names on one meta group and still got 10058 (B8). Card 06.

>

The **meta group is not per account**. T2 has one meta group with three JetStream accounts on (A1). So a dark region freezes creates in every account at the same moment **INFERRED**. An account is a wall for data, not for availability.

>

On T5 the leaf binds LB , so a subject crosses the link, and **each** meta group's ODOMETER stores the publish (D5). Neither can refuse the other's name: two same-named shadow streams hold 1 / 2 (D6).

READ A COUNT BY ITS SHAPE, NOT BY ITS NUMBER

COUNT	WHERE	WHAT IT MEANS
1 / 1	one meta group, shared account — A26 C13 F13 G7	One stored copy, read from both sides. The far read crosses the WAN (C13a).
1 / 1	a meta group per site, shared account over the leaf — D5	Two stored copies of one send. Double capture (D5a).
1 / 0	an account per region, any shape — A8 D18 H6	The other region's account never sees it.
1 / 0	T1, shared account, no link — T1f	Nothing connects the two sites at all.

PRO

>

An **account per region behaves the same on every shape**. Own names, own data, 1 / 0 (A6 A7 , D18 , H6).

>

A shared account over a gateway gives one global name and stores one copy (A4 , G7a).

CON

>

A **shared account over this lab's leaf captures twice, silently** (D5 , D6).

>

Over a gateway, AU's **stream info ODOMETER** returns za's stream (A5). It looks local. It is not.

>

Per-region accounts do not share data. Sharing needs a mirror or an export (cards 10, 11).

06

BOUNDARIES — DOMAINS AND LEAF LINKS

A domain names a JetStream. It is not a wall.

`jetstream.domain` gives one JetStream a name and an API prefix, `$JS.<domain>.API`. It does **not** decide which servers vote together: the links and the **system account** (`$SYS`) decide that. A domain string alone does not make a meta group.

DECISION

Inside one supercluster, use **one domain name everywhere, or none**. Across a leaf link, a domain per side is how you name, and reach, each JetStream. **Never use a domain as a security boundary.**

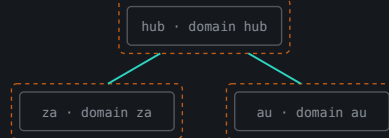
THREE RIGS — WHAT THE DOMAIN DID, AND WHAT THE LINK DID

B · GATEWAY + TWO DOMAIN NAMES
INVALID — B6 G19-G22



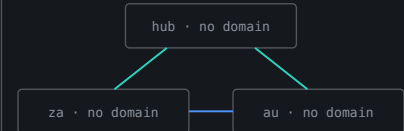
still ONE meta group of 6 — B3
only 1 of 2 sides elects — B1 B2
place on the blind side: 10005 — B4
which side goes blind: a coin toss
— G19a

T5 / H · LEAF, A DOMAIN EACH
\$SYS NOT CARRIED OVER THE LEAF



3 meta groups — D1 H1
a stream name alone: fails — H7
`$JS.<domain>.API`: crosses — D8 H11

G PHASE 1 · LEAF, NO DOMAIN
\$SYS NOT CARRIED OVER THE LEAF



2 meta groups, 3 / 6 / 6 — G1 G2
the hub stays its own JetStream
cause: \$SYS not carried (documented)

DOCUMENTED IN THE NATS DOCS — NOT TESTED ON THIS RIG

a leaf that carries \$SYS and sets no domain, to a clustered hub, extends the hub's JetStream: one meta group.
a domain on the leaf side stops that extension. a domain may change across a leaf, never inside a supercluster.
one domain name on every server of a supercluster, or none, is valid.
a domain names a JetStream and its API. the links and \$SYS decide which servers vote together.

MECHANISM

Routes and gateways join servers into one JetStream with one meta group. A leaf link joins the JetStream on each side **only** when it carries the system account and neither side sets a different domain **DOCUMENTED**.

This lab never carries **\$SYS** over a leaf. With a domain per side, that gave three JetStreams (**D1** , **H1**). With no domain at all, the hub still stayed separate (**G1** , **G2**). So in this lab the missing **\$SYS** link is enough on its own **INFERRED**.

A stream name resolves **in the local domain** (**H7a**). To reach another JetStream, name its API: **external.api** = **\$JS.za.API** (**D8** , **H11**). **external.api** names a domain, not an account (**H11a**).

Leave out **external.api** and a mirror either copies nothing (**D7**) or silently copies a **local** stream of the same name (**D9a**). No error either way (**D7a**).

WHEN IS A DOMAIN VALID, AND WHAT DOES IT DO?

WHERE	DOMAIN SETTING	RESULT	EVIDENCE
one supercluster (gateway)	none	valid: one JetStream	A1 C2 F2
one supercluster (gateway)	one name everywhere	valid: one JetStream, named	DOCUMENTED
one supercluster (gateway)	different names	broken: one side blind, 10005	B1 – B6 G19 – G22
leaf, \$SYS not carried	a name per side	separate JetStreams, reached by API	D1 D8 H1 H11
leaf, \$SYS not carried	none	separate JetStreams	G1 G2
leaf, \$SYS carried, hub clustered	none	extends the hub's JetStream	DOCUMENTED
leaf, \$SYS carried	a name per side	separate JetStreams	DOCUMENTED

PRO — USE A DOMAIN WHEN

You need to **reach another JetStream by name**: **\$JS.<domain>.API** , or **external.api** on a mirror (**D8** , **H11**).

You want a leaf side to keep its own JetStream even if **\$SYS** is carried **DOCUMENTED**.

CON

Not a security boundary. Anyone with the account can name the other API and read it, as **D8** and **H11** do on purpose.

Different names inside one supercluster break it at start-up, and which side breaks is a coin toss (**B1** , **G19a**).

07

LINKS — T2 / A, THE GATEWAY

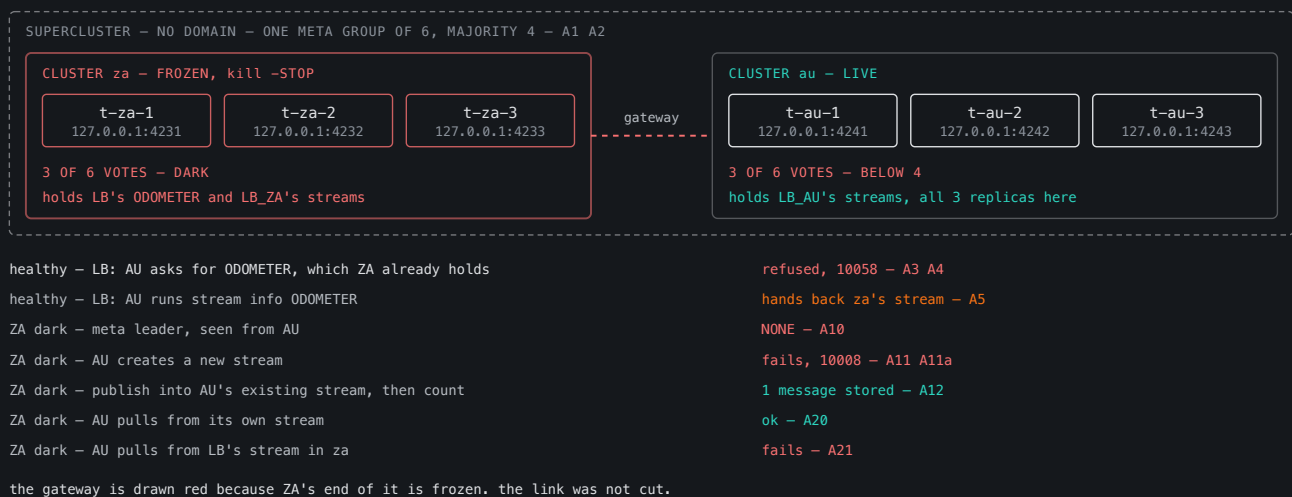
A gateway buys one name and sells your survival

A **gateway** joins two clusters into one **supercluster**. With JetStream on every server and no domain, the six servers form **one JetStream with one meta group**. One set of stream names per account, and one majority, held across the sea.

DECISION

Use a gateway when you want **one stream name per account, everywhere**, and you can add a third site that votes (card 08). Do not use a plain two-region gateway if either region must keep creating streams while the other is dark.

THE SHAPE, AND WHAT IT DOES WHEN ZA GOES DARK



MECHANISM

- › A gateway joins **clusters** into one supercluster. With JetStream on all six servers and no domain, they elect **one** meta group of six ([A1](#) , [A2](#)). One domain name on all six gives the same one JetStream, with a name [DOCUMENTED](#) . Different names break it (card 06).
- › Creating a stream needs 4 of 6. Each region holds 3. With ZA dark there is no meta leader ([A10](#)) and every create fails ([A11](#)). The lab froze ZA only. AU dark is the same arithmetic [INFERRED](#) .
- › The meta group is not per account ([A1](#) : one group, three JetStream accounts). So the freeze hits every account at once [INFERRED](#) .
- › A stream's replicas stay in one cluster ([A22](#) – [A24](#)). AU's streams keep their own majority, so a publish into them is stored ([A12](#)). Card 03.
- › After ZA comes back, the group is 6 again and nothing is lost ([A13](#) , [A14](#)).

PRO

- › **One name means one stream, in one account, everywhere.** No second [ODOMETER](#) by accident ([A4](#)).
- › A mirror in the other region copies with **no** [external.api](#) ([A15](#) , [A16](#) , [A17a](#)). There is one JetStream to look in.
- › Any client can place a stream in either region with `--cluster` ([E13](#) , [E14](#)). Card 04.

CON

- › **Two regions of three cannot out-vote each other.** Lose one and nobody can create, update or delete a stream.
- › A plain [nats pub](#) is a core publish and exits 0 either way. Count the stream to know ([A12](#)).
- › Streams stored in the dark region cannot be read ([A21](#)). AU's [stream info](#) on a za stream looks local; it is not ([A5](#)).

WHAT THE DRAWING DOES NOT SHOW

- › This is a **frozen region**, not a split. In a real WAN cut both sides stay alive. A 3 / 3 split leaves neither side a majority; an uneven split can leave one side a majority. Two sides can never both hold a majority of one group [INFERRED](#) . That rig is not built here.
- › The [leader](#) field in `/jsz` goes stale for a while after a majority dies ([A10a](#)). Wait for it; never read it once.

VERDICT

A two-region gateway is **one JetStream wearing two coats**. Add a third site that votes (card 08), or accept a change freeze when either region is dark.

08

QUORUM — T3 / C AND T4 / F, THE ARBITER

An arbiter site buys the vote back

Two regions of three cannot out-vote each other. **Three sites can.** Add a third cluster on the same gateway that votes and stores almost nothing. The meta group grows to 9, and either region alone can go dark.

DECISION

If you want one stream name across regions **and** creates during a region outage, pay for a third site. A three-node arbiter is the safe size. Keep clients off it.

T4 — NINE SERVERS, THREE SITES, ONE META GROUP — AND AU GOES DARK

SUPERCLUSTER — NO DOMAIN — ONE META GROUP OF 9, MAJORITY 5 — F1 F2

CLUSTER za — REGION

t-za-1

t-za-2

t-za-3

3 OF 9 VOTES
holds ZA's data

CLUSTER arb — THIRD SITE, :4540

t-arb-1

t-arb-2

t-arb-3

3 OF 9 VOTES — THE TIE BREAK
stores nothing unless placed — F20a

CLUSTER au — FROZEN, kill -STOP

t-au-1

t-au-2

t-au-3

3 OF 9 VOTES — DARK
AU's streams are dark with it

AU dark — 6 of 9 left: is a leader elected?

AU dark — ZA creates a new 3-replica stream

the other way — ZA dark: AU elects, creates, stores

AU dark + one arbiter node dark — 5 of 9, exactly 5

one more node dark — 4 of 9

4 of 9 — publish into an existing stream, then count

after each recovery — meta group size

the gateway to au is drawn red because au is frozen. the link was not cut.

yes — F3

ok — F4

yes / ok / 1 — F22 F23 F24

still elects and creates — F5 F6

NONE; create fails — F7 F8

1 message stored — F9 F21

9 — F10 F25

- › The meta group counts **servers**, not sites (card 02). Nine need five. Either region dark leaves six (**F3** , **F22**).
- › The arbiter only has to be reachable over the gateway and vote. It holds no copy, KV or consumer unless one is placed on it (**F20a** , **C20a**). Card 04.
- › A stream keeps its own group. Even with no meta leader, a publish into an existing stream whose replicas are alive is stored (**F9** , **F21**). Card 03.
- › The arbiter does not copy data across regions. Every replica of a stream stays in one cluster (**F17**). A stream stored in the dark region is still dark (**INFERRED**); a read was not measured on T4.

- › **One stream name per account, and creates with either region dark.** The only shape here that gives both, with two regions plus a voting third site.
- › The third site is small. It is a vote, not a data centre (C20a).
- › Recovery is automatic (F10 , F25).

- › **You need a real third place.** Three zones in one region do not survive that region's outage (**INFERRED**).
- › The meta group spans the WAN, so every create waits for it (**INFERRED**). Latency was not measured.
- › A one-node arbiter has no slack (**C12**). A client on its port stores data there (**C10** , **C11**).

Buy the third site, or give up one stream name across regions.
Without a third site, card 07's freeze stays.

09

LINKS — T5 / D AND H, HUB AND LEAF

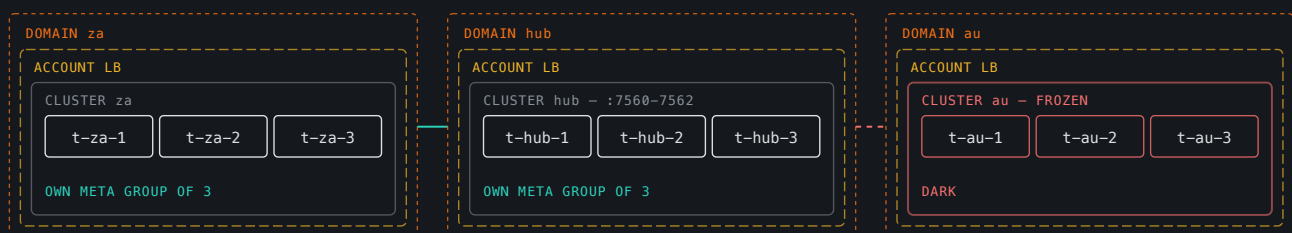
This lab's leaf link gives each region a vote, and a double copy

Each region hangs off a 3-node hub on a **leafnode link**. As this lab builds it, **\$SYS** is **not** carried over the leaf and each side sets its own domain. The result is **three JetStreams**, each with its own meta group of 3.

DECISION

Use this shape when **each region must keep creating streams alone**. Accept that one stream name no longer means one stream, and write every shared copy down on purpose.

T5 — THREE JETSTREAMS, AND WHAT THEY DO WHEN AU GOES DARK



each leaf remote binds LB only. \$SYS is not carried over the leaf.

healthy — meta groups, and their sizes

LB — both regions create ODOMETER

LB — one publish in ZA, then count both

AU dark — ZA: leader, create, store

whole hub dark — both regions create and store

mirror of an au stream, name only

the same, with external.api \$JS.au.API

LB_ZA, LB_AU — no leaf remote on T5

3; 3 / 3 / 3 — D1 D2

ok, in za / au, no 10058 — D3 D4

1 / 1 STORED, TWO copies — D5 D5a

yes / ok / 1 — D24 D25 D26

ok — D10-D13 D23

0 copied, no error — D7 D7a

2 copied — D8

six islands — D21 D22 D22a

MECHANISM

- › Two settings made three JetStreams here: `$SYS` is not carried over the leaf, and each side sets a domain (`D1`). With no domain at all the hub still stayed separate (`G1` , `G2`), so the `$SYS` setting did the separating (`INFERRED`). Card 06.
- › This is not the only way a leaf can work. A leaf that carries `$SYS` and sets no domain extends a clustered hub's JetStream instead (`DOCUMENTED`). Not tested here.
- › Each region's meta group is 3 servers at home, quorum 2 (`D2`). Another site going dark changes nothing (`D24 – D26` , `D10 – D13`).
- › A leaf remote binds **one** account (`D22a`). The lab bound `LB` , so its subjects cross the link, and each JetStream stores its own copy (`D5`). Nobody can refuse the name (`D3` , `D6`).
- › A stream in another domain is reached through its API: `external.api = $JS.au.API` (`D8`). Without it a mirror copies nothing, or silently copies a **local** stream of the same name (`D7a` , `D9a`).

VARIANT H — THE SAME SHAPE, AN ACCOUNT PER REGION

- › A second leaf remote per region server binds `LB_ZA` or `LB_AU` to the hub. Three JetStreams still (`H1`). One publish in `LB_ZA` reads `1 / 0` (`H6`). The hub holds three namespaces, not one shared one (`H9a`).
- › With the whole hub gone, a region still creates and stores (`H12 – H14` , `H16`). **A region dark was not run for H.** What H buys is a live subject feed to the hub, and nothing more (`H15`).

PRO

- › **Each region creates and stores alone** (`D25` , `D26`).
- › The hub is a reader, not a dependency. Kill it and both regions carry on (`D10 – D13` , `H14a`).
- › No shared majority, so a region can be added without a re-vote (`INFERRED`).

CON

- › **A shared account over this leaf captures twice, silently** (`D5` , `D6`).
- › Nothing is shared until somebody writes it down (`D15`). A mirror is a real thing to build and watch (card 10).
- › A name-only mirror can copy the wrong stream with no error (`D9a`).

VERDICT

For two regions that must each keep working, start here — with an account per region, and every shared stream a written mirror.

10

COPYING — MIRRORS

A mirror is the second copy — and it can copy the wrong stream

A mirror names a source stream. The name is looked up **in the asking side's own JetStream and own account**. If the source is anywhere else, the mirror must say where. If it does not, it is still accepted — and copies the wrong thing, or nothing.

DECISION

Every mirror whose source is in another JetStream sets `mirror.external.api`. Treat a mirror that has never been tested by counting as a **bug**, even when it is green.

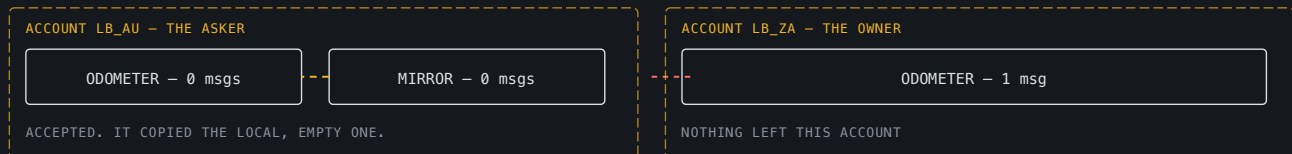
THE SAME CONFIG, ONE LINE APART

THE RIGHT WAY, ACROSS THIS LAB'S LEAF — NAME THE OTHER DOMAIN'S API



`external.api = $JS.au.API: 2 copied — D8 (H11 the same, hub to za). without it: 0, no error — D7 D7a H10`

THE TRAP — A NAME ALONE CANNOT CROSS AN ACCOUNT



`the mirror was NOT refused, and copied 0 of 1 messages — E17 E18 E18a`

`over a gateway there is one JetStream, so a name alone is enough: copied, no external.api — A15 A16 A17a`

MECHANISM

>

A mirror is a stream that copies another stream, one way. The copy is read-only.

>

A bare source name is looked up **in the asking side**: its own JetStream and its own account ([H7a](#) , [E18a](#)). Card 05 is the same fact seen from the other end.

>

[mirror.external.api](#) tells the mirror which JetStream to ask. It names a **domain**, not an account ([H11a](#)). No gateway, no shared vote.

>

No setting lets a mirror cross an **account**. For that, export the subject (card 11).

>

The lab proves each copy by **counting messages on both sides**, never by the mirror's status — the wrong mirror reports healthy too ([D7a](#) , [E18a](#)).

WHERE THE SOURCE IS, AND WHAT THE MIRROR NEEDS

THE SOURCE STREAM IS	THE MIRROR NEEDS	PROOF
in the same JetStream and account (gateway)	its name only	A15 A16 A17 A17a
in another domain, over this lab's leaf	external.api = \$JS.<domain>.API	D8 H11
... and the name only is given	0 copied, or a local stream copied	D7 H10 D9 D9a
in another account	not possible — copies the asker's own	E17 E18 E18a

PRO

>

Sharing without merging. Each region keeps its own vote and still gets the other's data ([D8](#)).

>

It is explicit. You can read the config and say what crosses.

>

The copy is local, so it can still be read when the source region is dark [INFERRED](#).

CON

>

The copy lags, and it stops moving while the source is dark [DOCUMENTED](#). Lag was not measured here.

>

You pay for the data twice, and for the link.

>

A wrong mirror is **silently wrong**, not loudly broken ([E17](#) , [E18](#) , [D9a](#)).

THE ONE TEST THAT CATCHES IT

>

Publish into the **source** only. Then count the mirror. If the count does not move, the mirror is bound to the wrong stream.

11

COPYING — T2 / E, EXPORT AND IMPORT

Export / import is the only sharing that is safe by design

An account shares nothing by accident. To let one account see another's messages, the owner **exports** and the guest **imports** — two edits, in two places, by two people.

DECISION

Share between tenants with **export / import** and an **import prefix**. Never by putting two tenants in one account, and never by hoping a domain will keep them apart (card 06).

ONE PUBLISH, AND WHERE IT LANDS

SUPERCLUSTER — NO DOMAIN — ONE META GROUP, ONE GATEWAY, TWO ACCOUNTS

ACCOUNT LB_ZA — THE OWNER

ODOMETER on `odo.>`

EXPORTS THE SUBJECT `odo.>` TO LB_AU
AND NOTHING ELSE AT ALL
STREAM INFO STILL REFUSED — E6

export

ACCOUNT LB_AU — THE GUEST

IMPORTED on `from_za.odo.>`

IMPORTS IT UNDER A PREFIX
ITS OWN ODOMETER IS A WHOLLY
DIFFERENT STREAM — E7 E8

publish once in LB_ZA
and the guest stored it too, across the gateway
but renamed — the guest's raw-subject stream stayed empty
the guest publishes that same subject back

the owner stored it, 1 — E1
1 — E2
0 — E3 E3a
the owner is unchanged — E4 E5

MECHANISM

- › The owner declares an **export** naming exactly the subjects it is willing to share. Nothing else is offered.
- › The guest declares an **import** and gives it a **prefix**. The message arrives on `from_zs.odo.>`, not on `odo.>`.
- › The prefix is the safety feature: the guest's own traffic and the imported traffic can never be confused, because they are on different subjects ([E3](#) , [E3a](#)).
- › Sharing a **subject** is not sharing a **stream**. The guest still cannot ask about the owner's stream at all ([E6](#)), and its own same-named stream is unrelated ([E7](#) , [E8](#)).
- › This was measured over a gateway, in one JetStream. The export rule is an account rule, so it does not depend on the link [DOCUMENTED](#).

PRO

- › **Nothing crosses unless two people agree.** The owner offers, the guest accepts.
- › The share is narrow. One subject pattern, not an account.
- › The prefix makes the origin of every message visible in its subject.

CON

- › **It is two edits in two configs.** Get either half wrong and it shares nothing [INFERRED](#).
- › It shares live subjects, not history. A new guest sees nothing sent before it imported [DOCUMENTED](#).
- › Every consumer on the guest side must use the prefixed subject.

WHY THIS CARD SITS NEXT TO CARD 10

- › Card 10 shows a mirror **failing silently** across an account wall ([E17](#) , [E18](#)). This card is the supported way to do what that mirror was trying to do. If you want data across an account, export it.

VERDICT

Export / import is the only account crossing that exists. It is deliberately small, and that is the point.

12

TRAPS — T6 / G, BOTH LINKS AT ONCE

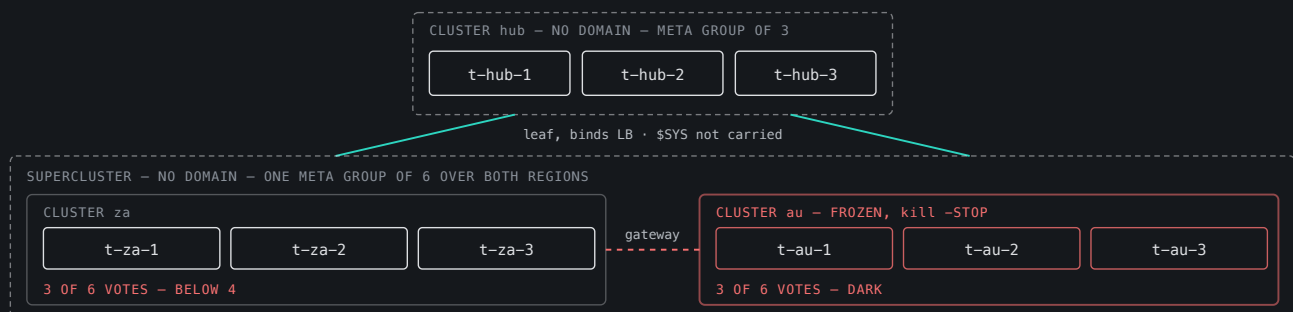
Both links at once is not both shapes at once

Add a hub leaf link to a gateway pair and it looks like belt and braces. It is not. **The gateway decides** (G24). You pay for a hub, and the two regions get no vote back.

DECISION

Pick **one** link type between the two regions. If they share a gateway, a leaf-linked hub cannot rescue them, however healthy it is.

A HEALTHY HUB, AND A REGION THAT STILL CANNOT CREATE



healthy — meta groups; sizes hub / ZA / AU

LB — AU asks for ODOMETER too

LB — one publish in ZA, then count both

AU dark — ZA's meta leader

AU dark — ZA creates a new stream

AU dark — publish into ZA's existing stream, then count

AU dark — the hub keeps a leader and still creates

whole hub gone — ZA's group size; both regions create

phase 2 — add a domain per site (hub, za, au)

phase 2 — group size; place on the blind side; the hub

2; 3 / 6 / 6 — G1 G2 G3

refused, 10058 — G4

1 / 1 = ONE copy — G7 G7a

NONE — G10

fails, 10008 — G11 G11a

2 stored — G12

and lends ZA nothing — G13 G14 G14a

6; ok / ok — G16 G17 G18

1 of 2 regions elects — G19 G19a G20

still 6; 10005; hub elects — G21 G22 G23

CONTINUES

The teal lines are real, and the blue line still decides. **A hub on a leaf has no vote in a gateway's meta group.**

Card 12 · 1 of 2

MECHANISM

- › The gateway still fuses **za** and **au** into one meta group of 6 (**G2** , **G3**). The leaf links do not undo that.
- › The hub stays its own JetStream of 3 with **no domain** set (**G1**). The leaf does not carry **\$SYS** , and a leaf joins a JetStream only when it does **DOCUMENTED** . Card 06.
- › With AU dark, ZA holds 3 of 6 and cannot reach 4 (**G10** , **G11**). The hub has a leader (**G13**), but it is not in that group, so it cannot lend a vote (**G14a**).
- › Phase 2 adds a domain per site. It breaks the way variant B breaks: one region blind, still one group of 6, **10005** on the blind side (**G19** – **G22**). The leaf does not rescue it. Only the hub, across the leaf, keeps its own leader (**G23**).

GETTING FROM T2 OR T6 TO T5 — WHY NOT IN PLACE, AND A PROPOSED WAY

- › **Not in place.** A leaf beside the gateway changes nothing (**G24**). A domain per site on the gateway breaks it (**G19** – **G22**).
- › **PROPOSED** Build T5 beside the old shape. Copy each stream across. Move clients region by region. Remove the gateway last. **Nobody has run this.**
- › If the goal is a third site that votes, join it with a **gateway**, not a leaf. That is card 08, and it is measured.

PRO — THE SHORT LIST

- › A hub gives **one more place to attach clients** that is not either region.
- › The hub keeps a working JetStream of its own, and can die with no effect on either region (**G16** – **G18**).

CON

- › **Every T5 property is gone.** **10058** is back (**G4**), one copy not two (**G7a**), and a dark region freezes creates (**G11**).
- › You pay for a third site that **cannot lend a vote** (**G14a**).
- › On a diagram it looks safer than a plain gateway. That is the trap.

VERDICT

A leaf link beside a gateway buys the two regions nothing. Do not build it.

13

RETRACTIONS — THE METHOD

What this lab got wrong, and corrected

A lab whose numbers only ever improve is not measuring. Six things here were wrong, were found, and were fixed. **Four of the fixes are now part of the rig**, so they cannot come back.

DECISION

Trust a measurement only when the rig **proved the precondition** and **counted the result**.
An exit code of 0 is not a measurement. A green health field is not a measurement.

TWO HABITS THE RIG NOW ENFORCES ON ITSELF

BEFORE — A FREEZE WAS ASSUMED

kill -STOP t-au-*

measure straight away

the cluster may still have been answering

NOW — A FREEZE MUST PROVE IT FROZE

kill -STOP t-au-*

wait_dark

then measure

A10 A11

BEFORE — A PUBLISH WAS ASSUMED

nats pub

exit code 0

a CORE publish. no JetStream ack. proves nothing.

NOW — A PUBLISH MUST BE COUNTED

nats pub

count stream_msgs

believe the number

A12 F21 D23 G12 H16

a check has an expected answer and passes only on an exact match. if the right answer cannot be stated in advance, it is recorded as a note, not a check.

THE CORRECTIONS, AND WHAT EACH ONE BECAME

WHAT WAS WRONG

A freeze was assumed. `kill -STOP` was sent and the measurement taken at once. A stopped process can still look alive for a moment.

A publish was read from an exit code. `nats pub` is a core publish and exits 0 either way.

Four summary cells claimed an answer nobody measured. Only `A21` reads a dark region's stream.

A run could fail and still exit 0. `check()` records a FAIL row and returns 0.

The last deck said "domains for who votes with whom". That gave the domain a job it does not do.

The last deck drew several figures wrong. T2 showed AU frozen, but the lab froze ZA. T2, T4 and E had a domain box with no domain set. T5 drew a 1-node hub and accounts it does not bind. T6 drew the hub as a domain in phase 1.

WHAT IT IS NOW

`wait_dark` polls until the region stops answering. The run fails if it never does.

Every publish claim is a **message count on the stream** — `A12`, `F21`, `D23`, `G12`, `H16`.

Those cells read **not measured**. The replica condition on "publish keeps working" was added with them.

`run-all.sh` counts FAIL rows and exits non-zero, naming the file.

A domain names a JetStream (card 06). In T5 the separate votes came from the lab's leaf config: `$SYS` not carried, and a domain per side (card 09).

Every figure is redrawn from the lab scripts: ZA frozen on T2 (card 07), plain frames where no domain is set, a 3-node hub binding `LB` (card 09), a plain hub in phase 1 (card 12).

PRO — WHAT THE CORRECTIONS BOUGHT

- › Every remaining claim is **reproducible from nothing** by one script.
- › A wrong result now stops the run instead of printing green.
- › Each claim now says whether it is measured, documented, inferred or proposed.

CON — WHAT THEY COST

- › Some questions lost their answer and now read **not measured**. That is still a gap.
- › `wait_dark` and message counting make every run slower.
- › Anything quoted from an earlier deck may be wrong. Re-check it against `REPORT.md`.

VERDICT

Every correction here became a rule in the rig or a redrawn figure, not a footnote. A rule in a document is a thing you forget.



CHOOSING

Pick the shape from the outage you must survive

Start from the sentence "during a region outage we must still be able to ...". The shape falls out of it. Nothing else in this deck decides for you.

THE SHAPES MEASURED, SIDE BY SIDE

SHAPE	META GROUPS	A STREAM NAME IS UNIQUE IN	REGION ALONE CAN STILL CREATE?	SITES	CARD
T1 no link	2, of 3 each	each region, each account	yes — nothing shared	2	01
T2 / A gateway	1, of 6	the account, everywhere	no — 3 of 6	2	07
T3 / C + 1-node arbiter	1, of 7	the account, everywhere	yes, no slack	3, one tiny	08
T4 / F + 3-node arbiter	1, of 9	the account, everywhere	yes, both ways	3	08
T5 / D hub and leaf	3, of 3 each	each domain, each account	yes	2 + a hub	09
T5 / H + an account per region	3, of 3 each	each domain, each account	hub gone: yes. region dark: not run	2 + a hub	09
T6 / G gateway AND leaf	2, of 3 and 6	the account, over both regions	no — and you paid for a hub	3	12

CHOOSE THIS LAB'S LEAF SHAPE (CARD 09) WHEN

- › A region must keep taking work while the other is dark. This is the usual answer for logistics.
- › You accept a mirror per shared stream, and can name what is shared (card 10).
- › You have no credible third site, or do not want to pay for one.

CHOOSE THE ARBITER SHAPE (CARD 08) WHEN

- › One stream name per account, everywhere, is worth more than regional independence.
- › You have a real third location, not a third zone in one region.
- › You can live with every stream change waiting on the WAN **INFERRED**.

RULES THAT HOLD WHICHEVER YOU PICK

- › **Say which of the three writes you mean** before agreeing any of this (card 03).
- › **Accounts for tenants, always.** A stream name is unique per meta group and account (card 05). A domain is not a wall (card 06).
- › **Cross an account only with export / import**, with a prefix (card 11). A mirror cannot do it and will not say so (card 10).
- › **Never add a leaf hub to a gateway pair** expecting quorum (card 12).



PROVENANCE

Where every number came from

A figure with no provenance becomes a stale constant. Every measured number in this deck comes from one run of one script, on one machine, on one day.

THE RUN	
WHAT	VALUE
When	2026-09-23 09:24 SAST
Server	nats-server v2.14.6
Client	nats CLI 0.4.0
Machine	Darwin 25.4.0 arm64, all servers on one host
Result	175 checks passed, 0 failed, 51 recorded observations
Produced by	demos/03-multi-cluster-and-accounts/lab/run-all.sh
Written to	REPORT.md and REPORT.html — both generated, never hand-edited

WHICH CARD USED WHICH CHECKS		
CARD	SHAPE	CHECKS CITED
01 the map	all	T1a – T1c A1 C1 F1 D1 G1
02 Raft and quorum	T1, T2, T3, T4, T5	A2 A10 C1 C12 F1 – F11 D2 D24 G10 T1g
03 three questions	T2 / A, and all	A11 – A21a C4 C6 D23 – D26 F4 F21 – F24 G11 G12 H16 T1h
04 placement	T4 / F, T3 / C	F13 – F20a C7 – C18 A9 A15 A19 A26 B4 E13 E14
05 accounts and names	T2, T5, E	A4 – A8 D3 – D6 D18 H6 G4 G7 E7 E8 T1d T1f
06 domains	B, T5 / D, H, G	B1 – B6 D1 D7 – D9a H1 H7 H11 G1 G19 G22
07 gateway	T2 / A	A1 – A25 E13 E14
08 arbiter	T3 / C, T4 / F	C1 – C12 C20a F1 – F25
09 leaf link	T5 / D, T5 / H	D1 – D26 H1 – H16 G1 G2
10 mirrors	T2, T5, H, E	A15 – A17a D7 – D9a H10 H11 E17 – E18a
11 export / import	T2 / E	E1 – E8
12 both links	T6 / G	G1 – G24
13 retractions	method	A10 – A12 A21 F21 D23 G12 H16

FOUR KINDS OF CLAIM, AND HOW EACH IS MARKED	
MARK	MEANS
a check ID, such as <code>A11</code>	Measured in the run above. The row in <code>REPORT.md</code> holds the expected and the actual value.
<code>DOCUMENTED</code>	Said by the NATS docs listed below. Not run in this lab.
<code>INFERRED</code>	Follows from measured facts and the docs, but nobody ran it. Treat it as a question for a later run.
<code>PROPOSED</code>	A step we suggest. Nobody has run it. It is a plan, not a finding.

THE DOCS BEHIND EVERY “DOCUMENTED” MARK — DOCS.NATS.IO	
PAGE	USED FOR
Leaf nodes › JetStream on leaf nodes	a leaf that carries <code>\$SYS</code> and sets no domain extends a clustered hub's JetStream; a domain on the leaf side keeps it separate; <code>\$JS.<domain>.API</code> reaches another domain
Clustering › JetStream clustering	one meta group; each stream and each consumer is its own Raft group; quorum is more than half; replicas 1, 3 or 5
Concepts › JetStream › Streams	placement by cluster or by tags (tags not tested here); mirrors and <code>external.api</code>
Reference › config › leafnodes › remotes	a leaf remote binds one local account; <code>urls</code> is failover for one link

WHAT THESE NUMBERS ARE NOT

- › **No timing in this deck is a production figure.** Every server ran on one laptop with no WAN between them. Latency and mirror lag were not measured (`D03-R7`).
- › **Every outage is `kill -STOP` on a whole region.** No split brain, no packet loss, no real WAN partition.
- › **A check ID is permanent.** IDs are never reused or renumbered, so a card citing `A11` keeps meaning the same measurement across runs.
- › If a card and `REPORT.md` disagree, the report is right. Re-run `lab/run-all.sh` and correct the card.